

AXI_LITE_READ_CHANNEL_DECODER



December 16, 2025

Jay Convertino

Contents

1 Usage	2
1.1 Introduction	2
1.2 Dependencies	2
1.2.1 fusesoc_info Depenecies	2
1.3 In a Project	2
2 Architecture	3
3 Building	3
3.1 fusesoc	3
3.2 Source Files	4
3.2.1 fusesoc_info File List	4
3.3 Targets	4
3.3.1 fusesoc_info Targets	4
3.4 Directory Guide	4
4 Simulation	5
4.1 iverilog	5
4.2 cocotb	5
5 Module Documentation	6
5.1 axi_lite_read_channel_decoder-v	7
5.2 tb_cocotb-py	9
5.3 tb_cocotb-v	13

1 Usage

1.1 Introduction

Main function of this core is allow axi lite read channel to pass data if there is an address match. Otherwise this will block all data downstream which would initiate a transfer. Once a valid address is presented it will connect in one clock cycle.

1.2 Dependencies

The following are the dependencies of the cores.

- fusesoc 2.X
- iverilog (simulation)
- cocotb (simulation)

1.2.1 fusesoc_info Depenecies

- dep
 - AFRL:utility:helper:1.0.0
 - AFRL:simple:holdbuffer:1.0.0
 - AFRL:bus:bus_addr_decoder:1.0.0

1.3 In a Project

Simply connect inline with the read address channel of the axi lite bus. Set the address to a valid value and the region to a power of two (makes the most sense anyways). and the decoder will only allow valid traffic to be passed.

```
axi_lite_read_channel_decoder #(
    .ADDRESS_WIDTH(ADDRESS_WIDTH) ,
    .BUS_WIDTH(BUS_WIDTH) ,
    .SLAVE_ADDRESS(SLAVE_ADDRESS) ,
    .SLAVE_REGION(SLAVE_REGION)
) dut (
    .connected(connected) ,
    .aclk(aclk) ,
    .arstn(arstn) ,
    .s_axi_araddr(s_axi_araddr) ,
    .s_axi_arprot(s_axi_arprot) ,
    .s_axi_arvalid(s_axi_arvalid) ,
```

```

.s_axi_arready(s_axi_arready),
.s_axi_rdata(s_axi_rdata),
.s_axi_rresp(s_axi_rresp),
.s_axi_rvalid(s_axi_rvalid),
.s_axi_rready(s_axi_rready),
.m_axi_araddr(w_m_axi_araddr),
.m_axi_arprot(m_axi_arprot),
.m_axi_arvalid(m_axi_arvalid),
.m_axi_arready(m_axi_arready),
.m_axi_rdata(m_axi_rdata),
.m_axi_rresp(m_axi_rresp),
.m_axi_rvalid(m_axi_rvalid),
.m_axi_rready(m_axi_rready)
);

```

2 Architecture

The only module is the `axi_lite_read_channel_decoder` module. It is listed below.

- **axi_lite_read_channel_decoder** Allow data to flow if valid address is applied (see core for documentation).

Please see 5 for more information.

3 Building

The `axi_lite_read_channel_decoder` core is written in Verilog 2001. They should synthesize in any modern FPGA software. The core comes as a fusesoc packaged core and can be included in any other core. Be sure to make sure you have met the dependencies listed in the previous section. Linting is performed by verible using the lint target.

3.1 fusesoc

Fusesoc is a system for building FPGA software without relying on the internal project management of the tool. Avoiding vendor lock in to Vivado or Quartus. These cores, when included in a project, can be easily integrated and targets created based upon the end developer needs. The core by itself is not a part of a system and should be integrated into a fusesoc based system. Simulations are setup to use fusesoc and are a part of its targets.

3.2 Source Files

3.2.1 fusesoc_info File List

- src
 - src/axi_lite_read_channel_decoder.v
- tb_cocotb
 - 'tb/tb_cocotb.py': 'file_type': 'user', 'copyto': '.'
 - 'tb/tb_cocotb.v': 'file_type': 'verilogSource'

3.3 Targets

3.3.1 fusesoc_info Targets

- default
 - Info: Default for IP intergration.
- lint
 - Info: Lint with Verible
- sim_cocotb
 - Info: Cocotb unit tests

3.4 Directory Guide

Below highlights important folders from the root of the directory.

1. **docs** Contains all documentation related to this project.
 - **manual** Contains user manual and github page that are generated from the latex sources.
 - **specs** Contains specifications for the bus.
2. **src** Contains source files for the core
3. **tb** Contains test bench files for iverilog and cocotb
 - **cocotb** testbench files

4 Simulation

There are a few different simulations that can be run for this core.

4.1 iverilog

iverilog is used for simple test benches for quick verification, visually, of the core.

4.2 cocotb

To use the cocotb tests you must install the following python libraries.

```
$ pip install cocotb
```

Each module has a cocotb based simulation. To install these locally use the following.cocotb

```
$ pip install —breaksystem-packages -e .
```

Then you must use the cocotb sim target. The targets above can be run with various parameters.

```
$ fusesoc run —target sim_cocotb AFRL:bus:  
  ↪ axi_lite_read_channel_decoder3:1.0.0
```

5 Module Documentation

- **axi_lite_read_channel_decoder** AXI lite read channel decoder
- **tb_cocotb-py** Cocotb python test routines
- **tb_cocotb-v** Cocotb verilog test bench

The next sections document the module in detail.

axi_lite_read_channel_decoder.v

AUTHORS

JAY CONVERTINO

DATES

2025/12/16

INFORMATION

Brief

Verify read address and contain holdbuffer interface. Valid address? Allow output till invalid address presented.

License MIT

Copyright 2025 Jay Convertino

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

axi_lite_read_address_decoder

AXI lite address recoder for read channel.

Parameters

ADDRESS_WIDTH	Width of the AXI LITE address port in bits.
BUS_WIDTH	Width of the AXI LITE bus data port in bytes.
DATA_BUFFER	Buffer data channel, 0 to disable.
TIMEOUT_BEATS	Number of clock cycles (beats) to count till timeout. 0 disables timeout.
SLAVE_ADDRESS	Array of Addresses for each slave (0 = slave 0 and so on).
SLAVE_REGION	Region for the address that is valid for the SLAVE ADDRESS.

Ports

connected	Core has established channel connection
aclk	Input clock
arstn	Input negative reset
s_axi_araddr	Slave read chanel input address.
s_axi_arprot	Slave read chanel input address protection
s_axi_arvalid	Slave read chanel input address is valid
s_axi_arready	Slave read chanel input is ready.
s_axi_rdata	Slave read chanel input data.
s_axi_rresp	Slave read chanel input data response.
s_axi_rvalid	Slave read chanel input data valid
s_axi_rready	Slave read chanel input is ready.
m_axi_araddr	Master read chanel output address.
m_axi_arprot	Master read chanel output address protection
m_axi_arvalid	Master read chanel output address is valid
m_axi_arready	Master read chanel output is ready.
m_axi_rdata	Master read chanel output data.
m_axi_rresp	Master read chanel output data response.
m_axi_rvalid	Master read chanel output data valid
m_axi_rready	Master read chanel output is ready.

INSTANTIATED MODULES

inst_addr_buffer

Buffer for the address

inst_addr_verify

Decoder for address bus.

inst_data_resp_buffer

If data buffer enabled, this holdbuffer will be generated.

tb_cocotb.py

AUTHORS

JAY CONVERTINO

DATES

2025/12/16

INFORMATION

Brief

Cocotb test bench

License MIT

Copyright 2025 Jay Convertino

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

FUNCTIONS

random_bool

```
def random_bool()
```

Return a infinite cycle of random bools

Returns: List

start_clock

```
def start_clock(  
    dut  
)
```

Start the simulation clock generator.

Parameters

dut Device under test passed from cocotb test function

reset_dut

```
async def reset_dut(  
    dut  
)
```

Cocotb coroutine for resets, used with await to make sure system is reset.

increment_test_read

```
@cocotb.test()  
async def increment_test_read(  
    dut  
)
```

Coroutine that is identified as a test routine. Read data from the device at the proper address and region.

Parameters

dut Device under test passed from cocotb.

increment_test_random_ready_read_data

```
@cocotb.test()  
async def increment_test_random_ready_read_data(  
    dut  
)
```

Coroutine that is identified as a test routine. Read data from the device at the proper address and region, randomize read data channel ready.

Parameters

dut Device under test passed from cocotb.

increment_test_random_ready_read_addr

```
@cocotb.test()  
async def increment_test_random_ready_read_addr(  
    dut  
)
```

Coroutine that is identified as a test routine. Read data from the device at the proper address and region, randomize address channel ready.

Parameters

dut Device under test passed from cocotb.

increment_test_timeout_no_answer

```
@cocotb.test()
async def increment_test_timeout_no_answer(
    dut
)
```

Coroutine that is identified as a test routine. Check if timeout will work as it should.

Parameters

dut Device under test passed from cocotb.

increment_test_random_ready_timeout_no_answer

```
@cocotb.test()
async def increment_test_random_ready_timeout_no_answer(
    dut
)
```

Coroutine that is identified as a test routine. Check if the timeout will work with random data read channel ready.

Parameters

dut Device under test passed from cocotb.

in_reset

```
@cocotb.test()
async def in_reset(
    dut
)
```

Coroutine that is identified as a test routine. This routine tests if device stays in unready state when in reset.

Parameters

dut Device under test passed from cocotb.

no_clock

```
@cocotb.test()
async def no_clock(
    dut
)
```

Coroutine that is identified as a test routine. This routine tests if no ready when clock is lost and device is left in reset.

Parameters

dut Device under test passed from cocotb.

tb_cocotb.v

AUTHORS

JAY CONVERTINO

DATES

2025/12/16

INFORMATION

Brief

Test bench wrapper for cocotb

License MIT

Copyright 2025 Jay Convertino

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE. BUS_WIDTH

tb_cocotb

```
module tb_cocotb #(
  parameter
  integer
  ADDRESS_WIDTH
  =
  32,
  parameter
  integer
  BUS_WIDTH
  =
  4,
  parameter
  [ADDRESS_WIDTH-1:0]
  SLAVE_ADDRESS
```

```

    =
    32'h44A20000,
    parameter
    [ADDRESS_WIDTH-1:0]
    SLAVE_REGION
    =
    32'h0000FFFF
)

output
wire
connected,
input
wire
aclk,
input
wire
arstn,
input
wire
[ADDRESS_WIDTH-1:0]
s_axi_araddr,
input
wire
[2:0]
s_axi_arprot,
input
wire
s_axi_arvalid,
output
wire
s_axi_arready,
output
wire
[BUS_WIDTH*8-1:0]
s_axi_rdata,
output
wire
[1:0]
s_axi_rresp,
output
wire
s_axi_rvalid,
input
wire
s_axi_rready,
output
wire
[ADDRESS_WIDTH-1:0]
m_axi_araddr,
output
wire
[2:0]
m_axi_arprot,
output
wire
m_axi_arvalid,
input
wire
m_axi_arready,
input
wire
[BUS_WIDTH*8-1:0]
m_axi_rdata,
input
wire

```

(

```

    [1:0]
    m_axi_rresp,
    input
    wire
    m_axi_rvalid,
    output
    wire
    m_axi_rready,
    input
    wire
    [ADDRESS_WIDTH-1:0]
    s_axi_awaddr,
    input
    wire
    [2:0]
    s_axi_awprot,
    input
    wire
    s_axi_awvalid,
    output
    wire
    s_axi_awready,
    input
    wire
    [BUS_WIDTH*8-1:0]
    s_axi_wdata,
    input
    wire
    [BUS_WIDTH-1:0]
    s_axi_wstrb,
    input
    wire
    s_axi_wvalid,
    output
    wire
    s_axi_wready,
    output
    wire
    [1:0]
    s_axi_bresp,
    output
    wire
    s_axi_bvalid,
    input
    wire
    s_axi_bready,
    output
    wire
    [ADDRESS_WIDTH-1:0]
    m_axi_awaddr,
    output
    wire
    [2:0]
    m_axi_awprot,
    output
    wire
    m_axi_awvalid,
    input
    wire
    m_axi_awready,
    output
    wire
    [BUS_WIDTH*8-1:0]
    m_axi_wdata,
    output
    wire

```



```

[BUS_WIDTH-1:0]
m_axi_wstrb,
output
wire
m_axi_wvalid,
input
wire
m_axi_wready,
input
wire
[1:0]
m_axi_bresp,
input
wire
m_axi_bvalid,
output
wire
m_axi_bready
)

```

AXI lite address recoder for read channel. Write channel lines included for cocotb cores to connect to.

Parameters

ADDRESS_WIDTH parameter integer	Width of the AXI LITE address port in bits.
BUS_WIDTH parameter integer	Width of the AXI LITE bus data port in bytes.
SLAVE_ADDRESS parameter [ADDRESS_WIDTH- 1:0]	Array of Addresses for each slave (0 = slave 0 and so on).
SLAVE_REGION parameter [ADDRESS_WIDTH- 1:0]	Region for the address that is valid for the SLAVE ADDRESS.

Ports

connected output wire	Core has established channel connection
aclk input wire	Input clock
arstn input wire	Input negative reset
s_axi_araddr input wire [ADDRESS_WIDTH- 1:0]	Slave read chanel input address.
s_axi_arprot input wire [2:0]	Slave read chanel input address protection
s_axi_arvalid input wire	Slave read chanel input address is valid
s_axi_arready output wire	Slave read chanel input is ready.
s_axi_rdata output wire [BUS_WIDTH* 8- 1:0]	Slave read chanel input data.
s_axi_rresp output wire [1:0]	Slave read chanel input data response.
s_axi_rvalid output wire	Slave read chanel input data valid
s_axi_rready input wire	Slave read chanel input is ready.
m_axi_araddr output wire [ADDRESS_WIDTH- 1:0]	Master read chanel output address.

m_axi_arprot output wire [2:0]	Master read channel output address protection
m_axi_arvalid output wire	Master read channel output address is valid
m_axi_arready input wire	Master read channel output is ready.
m_axi_rdata input wire [BUS_WIDTH* 8- 1:0]	Master read channel output data.
m_axi_rresp input wire [1:0]	Master read channel output data response.
m_axi_rvalid input wire	Master read channel output data valid
m_axi_rready output wire	Master read channel output is ready.

INSTANTIATED MODULES

dut

Device under test, axi_lite_read_channel_decoder